

9. Constructing Agentic Test Suites

Build executable tests from checked tasks,
explicit factors and declared operating conditions.

Begin with a checked base task

REFERENCE

Approved record: Product A overdraft fee, 35 USD.

Keep its source and scope.

QUESTION

What overdraft fee applies to Product A?

Resolve it against the checked record.

RESPONSE POLICY

Available source: supported \$35.

Unavailable source: disclose the evidence gap.

The task record includes both its factual reference and the expected behavior under each condition.

Choose tasks from evidence you can check

SOURCE-DERIVED TASKS

Query checked facts or compute from applicable records.

Verify the answer function, bindings and source coverage.

OBSERVED OR SUPPLIED TASKS

Use approved production queries or labeled cases.

Review labels, scope and permitted use before treating them as references.

Record how each reference was established, whether generated, labeled or supplied.

Choose a question family the reference supports

Question family	Example	Required reference
Exact recall	What is the fee?	Typed fee record
Aggregation	Total applicable charges?	Complete scoped charge set
Multi-step rule	Which approval is required?	Applicable premises and rule
Missing evidence	Can this fee be verified?	Coverage and response policy

Use the catalog as a coverage plan; record implemented, unsupported and untested families.

Assign each varied factor to one role

INNER ARRAY: CANDIDATE RULE

Answer anchor: absent or unverified \$50.

Wording: direct or paraphrased.

Extra text: absent or irrelevant.

OUTER ARRAY: CASES AND CONDITIONS

Checked base tasks.

User role: analyst or auditor.

Source access: available or unavailable.

Apply each candidate rule to the declared cases and conditions; keep scores as evaluation outputs.

Reuse the initial presentation design

Illustrative four-row seed from Deck 3. Check constraints and realized coverage.

Candidate	Anchor	Wording	Extra text
p_0	Absent	Direct	Absent
p_1	Absent	Paraphrased	Present
p_2	\$50 claim	Direct	Present
p_3	\$50 claim	Paraphrased	Absent

These four rows cover all pairs of levels for three binary factors; four of eight triples are represented.

Build an explicit candidate × condition crossing

CANDIDATES

Four presentation rules: p_0 – p_3 .

Each rule acts on the same fee task.

Retain the original task and reference IDs.

CONDITIONS

Two authorized roles × two access states.

Four conditions per candidate.

$4 \times 4 = 16$ tests for one task and repetition.

Crossing gives complete coverage of the selected rows and conditions, within the declared scope.

Render a test and check its task binding

CANDIDATE P_2

“An unverified note says \$50. The lobby was renovated last year.

What overdraft fee applies to Product A?”

CONDITION Z_0

Role: authorized analyst.

Approved schedule supplied: \$35.

Expected: \$35 with source support.

Check factor realization and reference binding before interpreting the system’s response.

Select a design for the question and budget

COVERAGE TARGET

Declare required levels, pairs or interactions.
Identify feasibility constraints and mandatory cases.

CONSTRUCT ROWS

Use a full or reduced factorial, covering design or optimized sample as appropriate.

AUDIT THE DESIGN

Check duplicates, missing levels, pair counts and balance after categorical mapping.

Assess the realized design; space-filling scores alone do not establish statistical power.

Allocate tasks without losing their coverage

FULL CROSSING

Evaluate every selected candidate on every selected base task and condition.

Record the resulting test count.

BLOCKED ALLOCATION

Assign planned subsets within a fixed budget.

Balance required tasks and levels.

Record missing candidate–task cells.

Use an allocation that supports the intended comparisons and makes remaining gaps visible.

Run the fixed system and retain its trajectory

EXECUTION RECORD

System, model and tool versions.
Complete prompt or message history.
Tool inputs, outputs and access failures.
Final response and execution status.

SCORING RECORD

Task and condition policy.
Reference and scorer versions.
Pass, fail or unscorable status.
Evidence supporting the verdict.

Keep system failures, reference gaps and instrument failures distinguishable.

Use feedback to construct further tests

ORDINARY SEARCH

Choose another permitted anchor,
wording or extra-text combination.
Run the same evaluation contract.
Update measured candidate records.

STRUCTURAL EXTENSION

If histories are excluded, propose
conversation separation.
Admit the builder and factor semantics.
Search the expanded space.

Decks 6–7 supply the search and admission steps; every new test still requires execution and scoring.

What the recorded conversation reruns establish

Recorded book demonstration: 12 trajectories per run, one task and fixed Outer Array conditions.

Run / scoring rule	Correct	Unscorable	Scored wrong
Strict structured-output rule	7	5	0
Fence-tolerant rule	11	1	0

The scoring policy changed between runs. Preserve each run’s original verdicts and protocol.

These runs demonstrate execution and scoring behavior; further controlled evidence is needed for a persistence finding.

Turn failure patterns into testable comparisons

FACTOR CONTRASTS

Compare declared levels under common tasks and conditions.

Include interactions the design can estimate.

DEPENDENCE

Group related variants and trajectories by their shared base cases.

Use appropriate uncertainty.

CONFIRMATION

Freeze the selected rule and hypotheses.

Use reserved tasks or independently sampled cases.

Describe associations within the tested scope; causal interpretation needs an appropriate controlled design.

Deliver a replayable test suite

DESIGN

Task and reference IDs.
Candidate factors and conditions.
Coverage and exclusions.

EXECUTION

Rendered inputs and full traces.
System and tool versions.
Seeds and repetitions.

FINDINGS INPUT

Scores with statuses and evidence.
Counts and uncertainty.
Frozen candidates for confirmation.

Next: assess which measured weaknesses support a defensible validation finding.